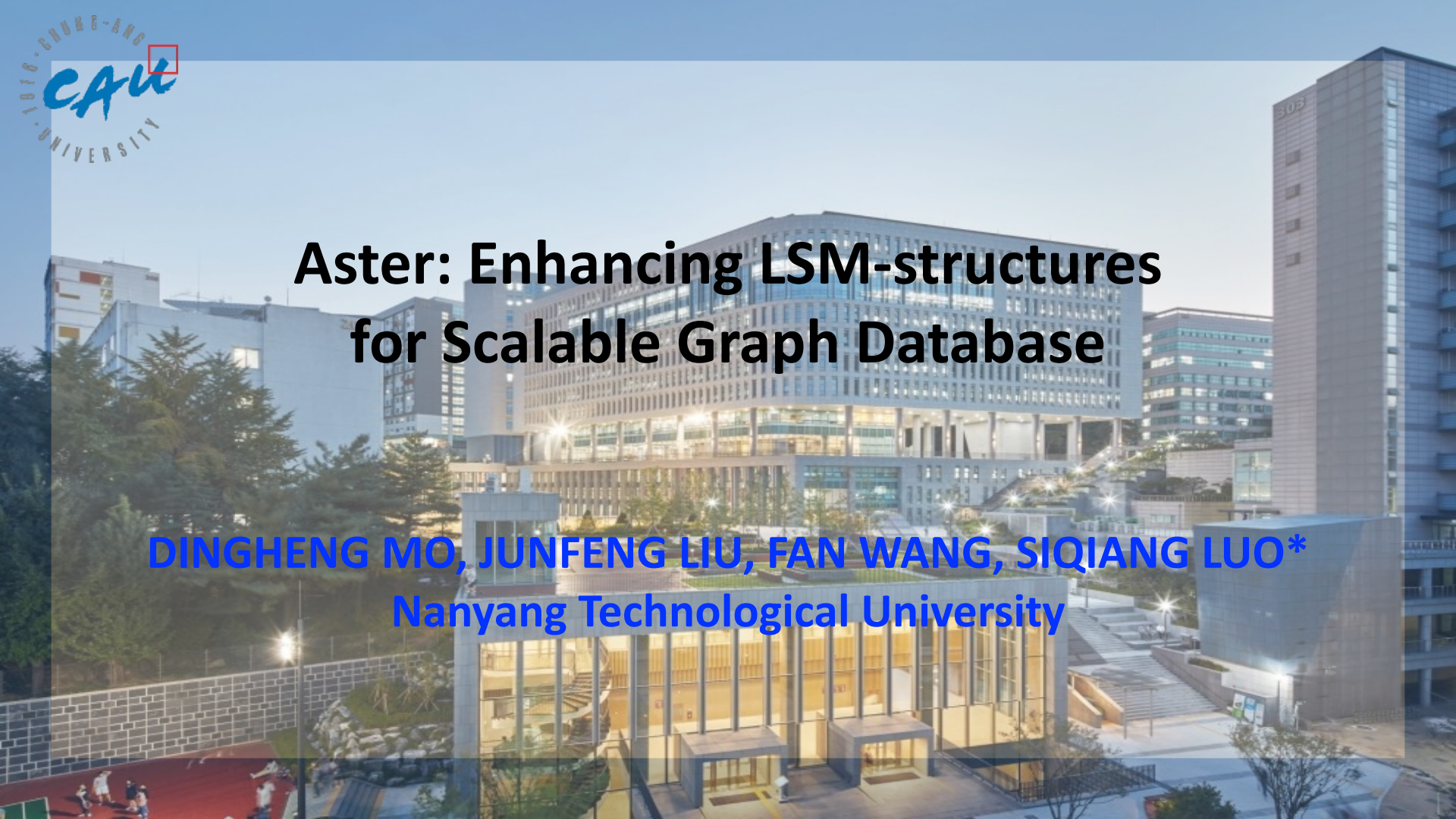
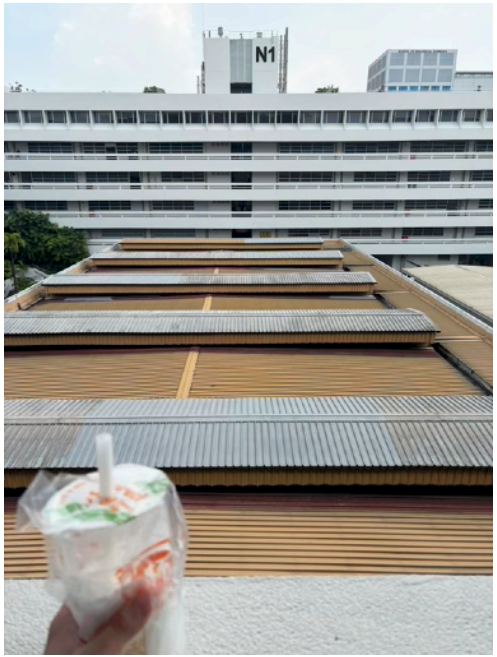


Aster: Enhancing LSM-structures for Scalable Graph Database

DINGHENG MO, JUNFENG LIU, FAN WANG, SIQIANG LUO*
Nanyang Technological University

Opening

- https://hi-gon.vercel.app/



INDEX

- Introduction
- Background: LSM-Tree
- Method: Poly-LSM
- Experiments & Summary
- Appendix

ASTER: Enhancing LSM-structures for Scalable Graph Database

DINGHENG MO, Nanyang Technological University, Singapore
JUNFENG LIU, Nanyang Technological University, Singapore
FAN WANG, Nanyang Technological University, Singapore
SIQIANG LUO*, Nanyang Technological University, Singapore

There is a proliferation of applications requiring the management of large-scale, evolving graphs under workloads with intensive graph updates and lookups. Driven by this challenge, we introduce POLY-LSM, a high-performance key-value storage engine for graphs with the following novel techniques: (1) POLY-LSM is embedded with a new design of graph-oriented LSM-tree structure that features a hybrid storage model for concisely and effectively storing graph data. (2) POLY-LSM utilizes an adaptive mechanism to handle edge insertions and deletions on graphs with optimized I/O efficiency. (3) POLY-LSM exploits the skewness of graph data to encode the key-value entries. Building upon this foundation, we further implement ASTER, a robust and versatile graph database that supports Gremlin query language facilitating various graph applications. In our experiments, we compared ASTER against several mainstream real-world graph databases. The results demonstrate that ASTER outperforms all baseline graph databases, especially on large-scale graphs. Notably, on the billion-scale Twitter graph dataset, ASTER achieves up to 17x throughput improvement compared to the best-performing baseline graph system.

CCS Concepts: • **Information systems** → **Graph-based database models**; **Data management systems**; **Information storage systems**; • **Theory of computation** → **Data structures design and analysis**.

Introduction

❑ Why Graphs?

- Natural way to represent relationships: things (nodes) connected by links (edges)
- Widely used across many domains

Social Graph

User-Item Graph

Knowledge Graph

Introduction

❑ Graphs Need a Database

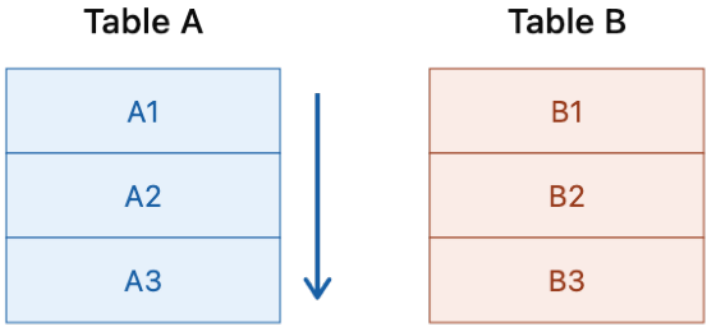
- A graph is data and data has to be stored somewhere: a database
- A database does two core things: **lookup (read)** and **update (write)**

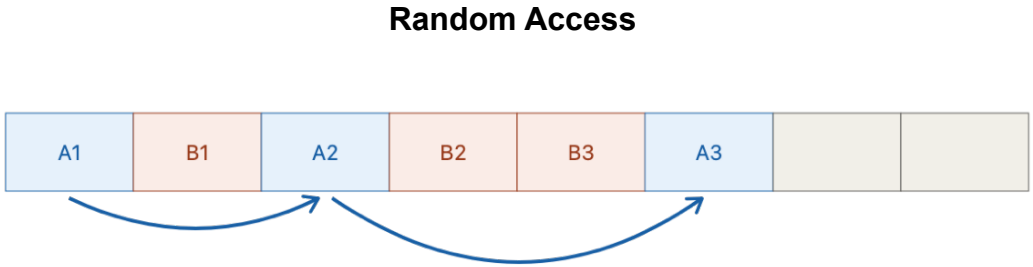
➡ Do we need a graph specialized database? **First, let's understand the cost**

Introduction

❑ Where Does the Cost Come From?

- Most of the cost comes from **random access** — reaching data scattered across the disk
- **Random access:** physically moving to where the data actually sits
- **Logically connected ≠ Physically close**



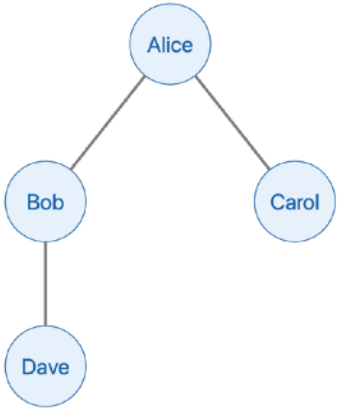


Introduction

❑ Why a Graph Specialized Database? — Neighbor Lookup Cost Explodes

- A graph's most common operation: **finding a node's neighbors**
- Traditional RDB → Neighbor lookup needs a **join** → Each hop = join → **Random access explodes**

➡ **We need a database built for how graphs are queried!**



stored as →

Users (nodes)

id	name
1	Alice
2	Bob
3	Carol
4	Dave

Friendships (edges)

from	to
1	2
1	3
2	4

Finding Bob's 2-hop neighbor Carol

Users	Friendships	Users	Friendships	Users
Bob (2)	(2, 1)	Alice (1)	(1, 3)	Carol (3)

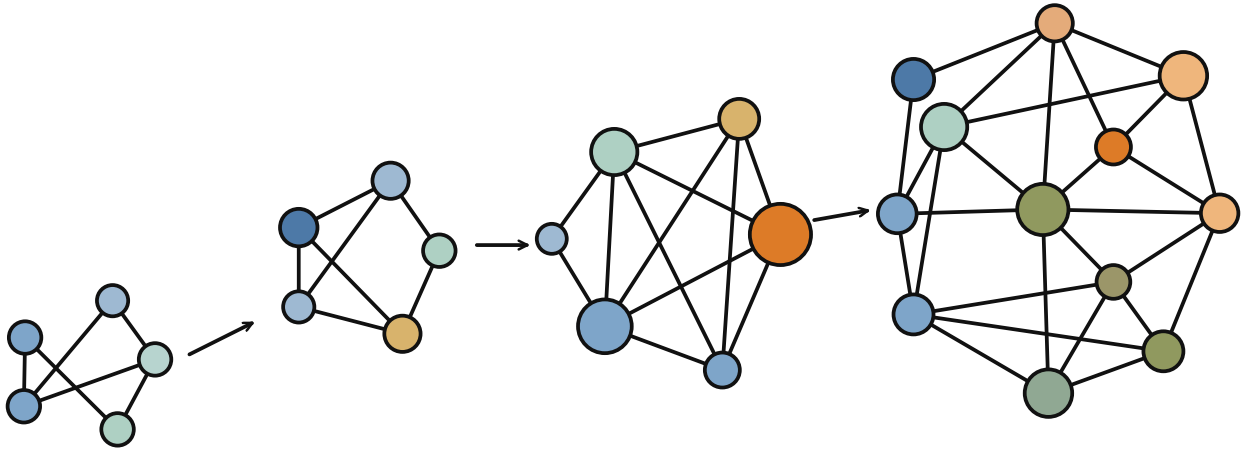
JOIN JOIN JOIN JOIN

4 joins

Introduction

❑ Another Consideration — Today's Graphs

- Real-World Graphs Keep Growing — billions of users, exploding data, constant queries
- e.g. social media: new follows and likes / fresh recommendations from latest activity
- So modern evolving graphs need **scalability** with **fast updates and lookups**

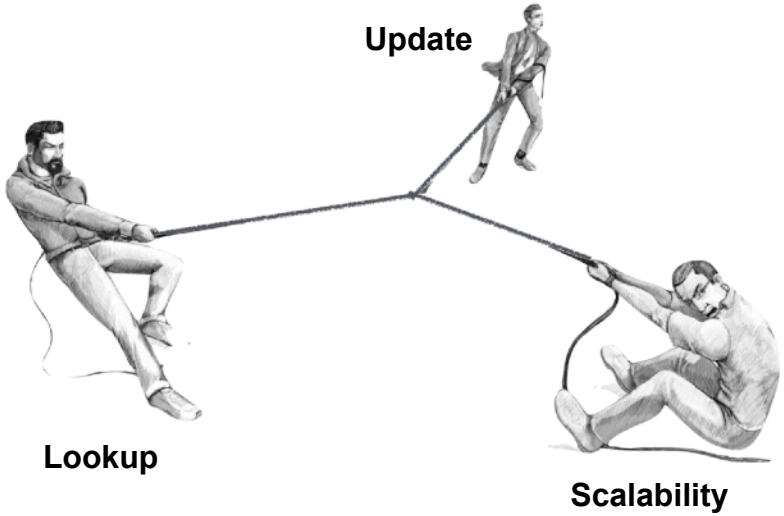


Introduction

❑ The Problem: No Single Design Does It All

- What modern evolving graphs need: **fast lookup** + **fast update** + **scalability**
- These three have trade-offs against one another

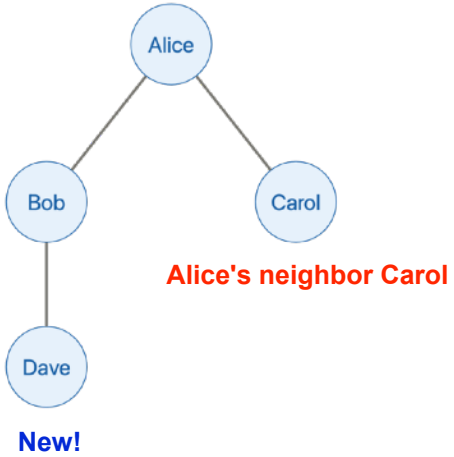
➡ **First, let's look at the existing storage structures**



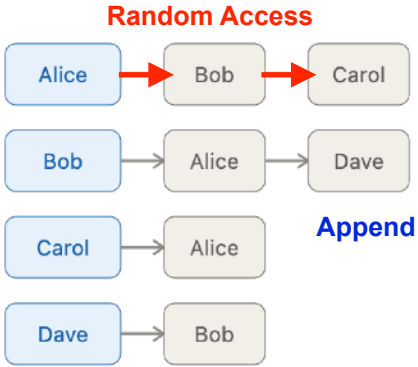
Introduction

❑ Two Existing Storage Structures — Linked-List & Relational Table

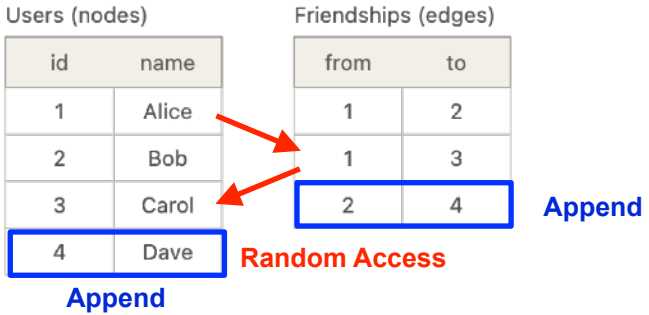
- **Linked-list:** each node links to its neighbors — used by Neo4j, the top graph DB
- **Relational Table**: nodes and edges in tables
- **Fast update** — just go to the end and append, **Slow lookup** — linked ≠ physically close → random access
- **Linear scalability** — more nodes means linearly increasing random access



Linked List



Relational Table



Introduction

❑ Existing Storage Structures vs. LSM-Tree (Log-Structured Merge)

- Linked-list & relational table: update is okay, but lookup is weak and scalability is limited
- **LSM-tree** is better and it comes in two forms: **node-based** and **edge-based**

➡ **Now, let's look into the LSM-tree**

Structures	Linked List	Relational Table	LSM-tree (Node -based)	LSM-tree (edge-based)
Update	★★★	★★★	★★	★★★★★
Lookup	★★	★	★★★★	★★★
Scalability	★★	★★	★★★★	★★★★

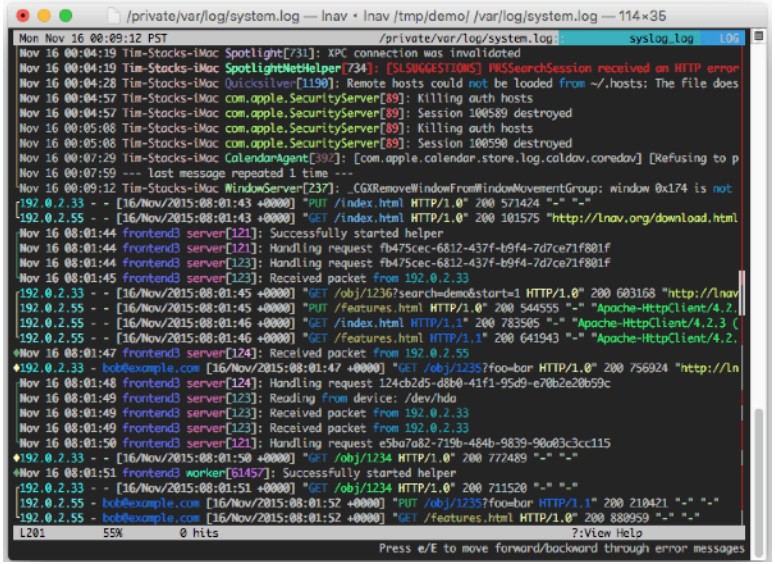
INDEX

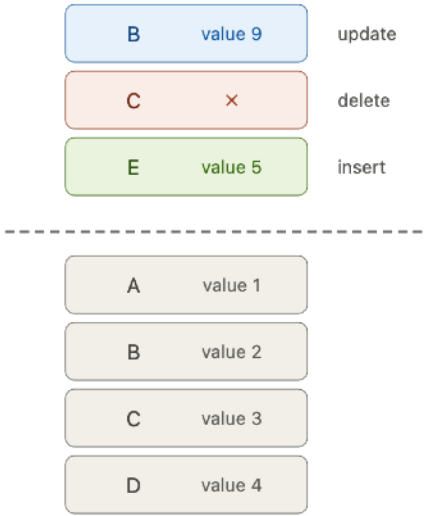
- Introduction
- Background: LSM-Tree
- Method: Poly-LSM
- Experiments & Summary
- Appendix

Background: LSM-Tree

❑ What is a Log-Structured Merge Tree? — Log-Structured Design

- A storage structure designed for fast updates
- "Log" as like logging things, not the math log (append-only records)
- Log-structured storage basic idea: **don't edit — stack writes, organize later**

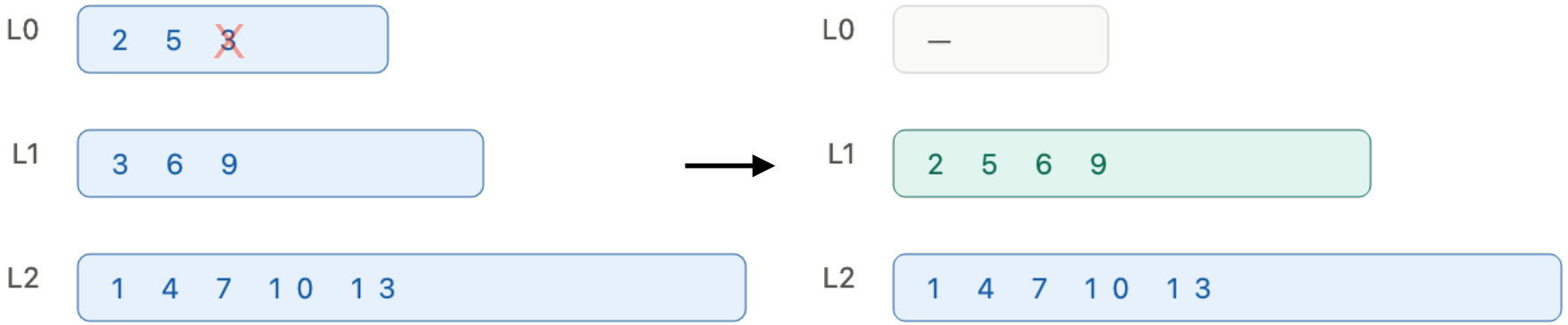




Background: LSM-Tree

❑ What is a Log-Structured Merge Tree? — Sorted, Merge-based, Leveled

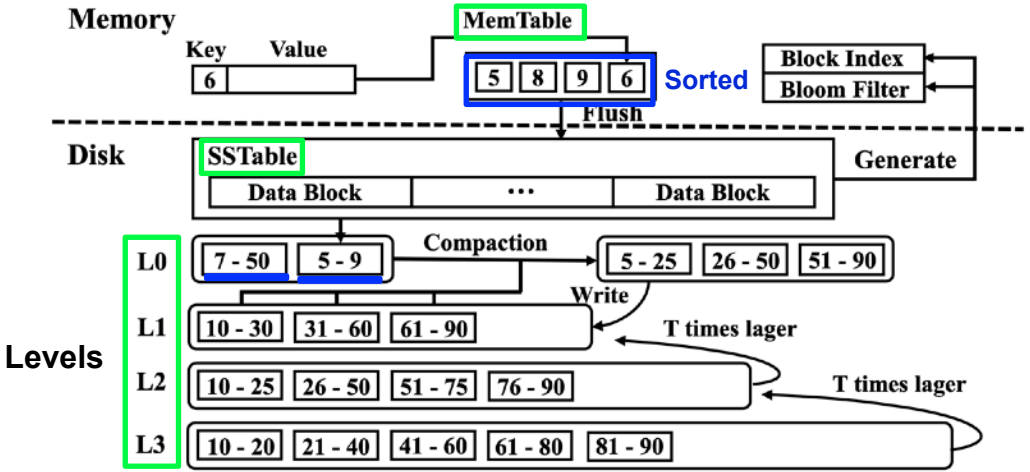
- LSM's main idea: **Keep the data sorted while merging (sorted & merge-based compaction)**
- **Sorted:** if needed data is contiguous, sequential access is possible, not random
- **Merge-based:** keeps sorted while removing old versions & tombstones
- **Leveled:** exponentially larger levels → costs grow logarithmically, not linearly
  - "Tree" is just a legacy name (original LSM used trees, but modern LSM doesn't)



Background: LSM-Tree

❑ LSM-Tree Structures — MemTable, SSTable, Levels

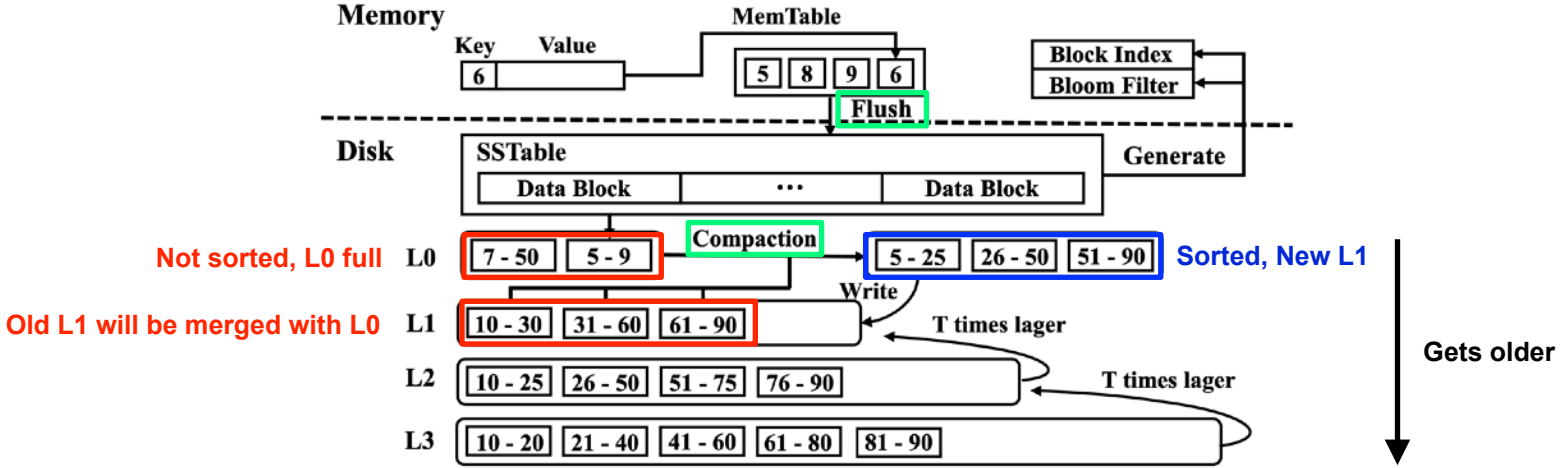
- **MemTable**: an in-memory buffer where new writes are first stacked, kept sorted by key
- **SSTable**: the immutable file on disk that a full MemTable is written into
  - Immutable: editing would break the sorted order
- **Levels**: SSTables are organized into levels (L0, L1, L2…), each level is ~T × larger
  - L0: each flushed SSTable is sorted, but L0 as a whole is not (SSTables overlap at L0)



Background: LSM-Tree

❑ LSM-Tree Operations — Flush & Compaction

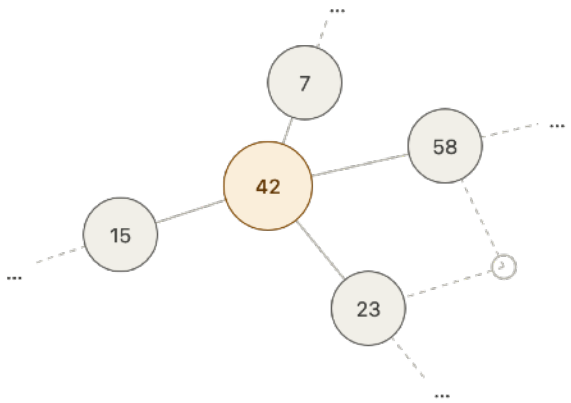
- **Flush**: when the MemTable fills, write it to disk as a **immutable SSTable**
  - No overhead of finding & rewriting old data → **fast update**
- **Compaction**: merge sort SSTables down into the next level, dropping old versions & tombstones
  - Data stays sorted and cleaned up across levels → **fast lookup**



Background: LSM-Tree

❑ How is a Graph Stored in a LSM-Tree?

- LSM-tree is key-value — **Key-Value**: each entry is one key and its value, no other columns
- In graphs, **key as node ID** and **value as its neighbors ID**
- **Node-based**: value has all neighbors (one entry per node)
- **Edge-based**: value has one neighbor (one entry per edge)



Node-based

key: 7	value: {42, ...}
key: 15	value: {42, ...}
key: 23	value: {42, ...}
key: 42	value: {7, 15, 23, 58}
key: 58	value: {42, ...}

...

Edge-based

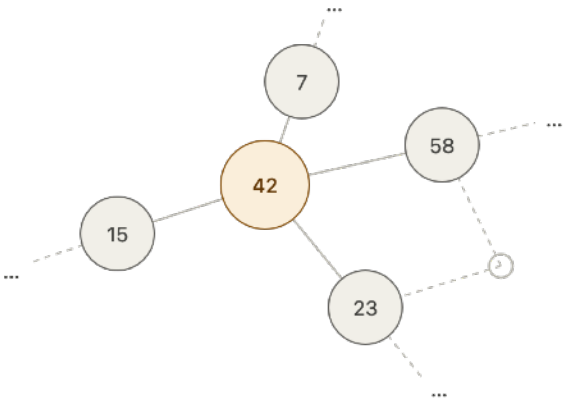
key: 7	value: 42
key: 15	value: 42
key: 23	value: 42
key: 42	value: 7
key: 42	value: 15
key: 42	value: 23
key: 42	value: 58
key: 58	value: 42

...

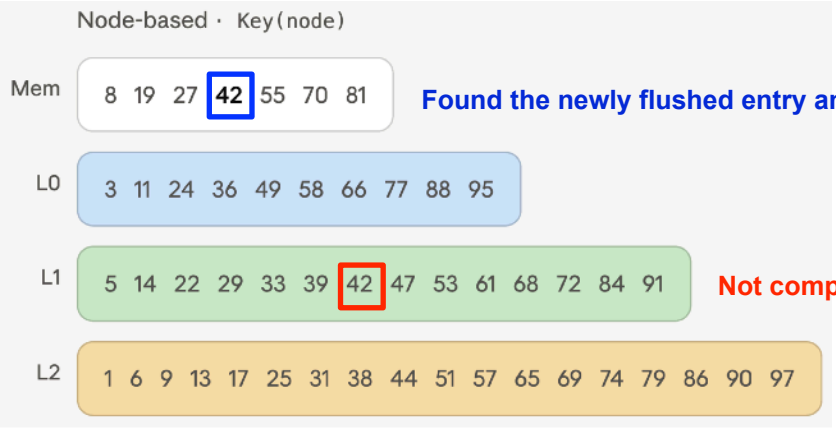
Background: LSM-Tree

❑ How does a Graph Operate in an LSM-Tree? — Node-based

- **Entry**: key is a node ID, and the value is the node's neighbor list
- **Lookup**: read the newest entry and stop
- **Update**: fetch and read the current list, add/delete the neighbor, write the new entry to the memtable
  - Old entries aren't deleted on update, only later when compaction merges them



Add node 42
42-{7, 15, 23, 58} appended to mentable
also, read-modify-write all the effected entries

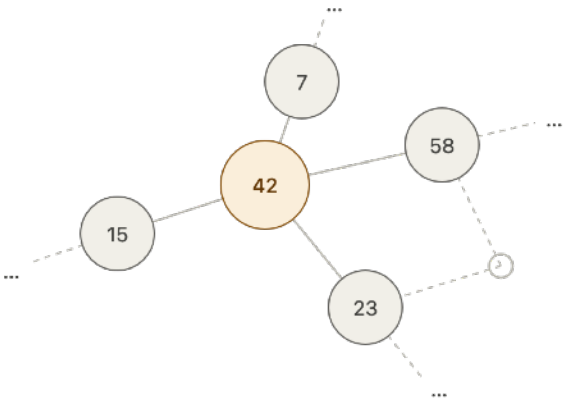


Find 42's neighbors

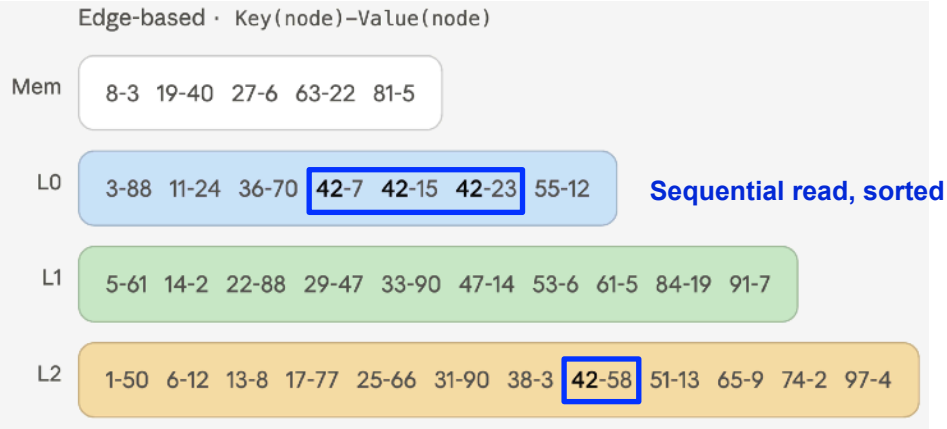
Background: LSM-Tree

❑ How does a Graph Operate in an LSM-Tree? — Edge-based

- **Entry**: key is a node ID, and the value is the node's one neighbor
- **Lookup**: scan every level to find all of the node's edges
  - Graphs mostly run neighbor queries, and same-key edges are clustered, so they're read sequentially
- **Update**: just append one edge at a time



Add node 42
42-7, 42-15, 42-23, 42-58
appended to meltable



Find 42's neighbors

Background: LSM-Tree

❑ Node & Edge Summary — Lookup vs Update

	Lookup	Update
Node	one entry has them all → fast	read-modify-write all the effected entries → slow
Edge	gather entries across levels → slow	append new entry right away → fast

Background: LSM-Tree

❑ Comparing LSM-Tree with the Two Existing Structures

- **Lookup**: levels are sorted, so neighbors are read sequentially, not one random access at a time → fast
  - Even edge-based(gathering entries), only jumps once per level — far fewer than jumping per neighbor
- **Update(edge-based)**: just appends — no finding or rewriting → fastest
- **Scalability**: levels grow ~10× (logarithmic) — random access only as many levels as there

Structures	Linked List	Relational Table	LSM-tree (node -based)	LSM-tree (edge-based)
Update	★★★	★★★	★★	★★★★★
Lookup	★★	★	★★★★	★★★
Scalability	★★	★★	★★★★	★★★★

Background: LSM-Tree

❑ Still Not Enough for Modern Evolving Graphs

- Modern evolving graphs need all three: fast update, fast lookup, scalability
- Even LSM-tree still has a **lookup–update trade-off**

➡ **Proposes Poly-LSM to close this gap**

Structures	Linked List	Relational Table	LSM-tree (node -based)	LSM-tree (edge-based)
Update	★★★	★★★	★★	★★★★★
Lookup	★★	★	★★★★	★★★
Scalability	★★	★★	★★★★	★★★★

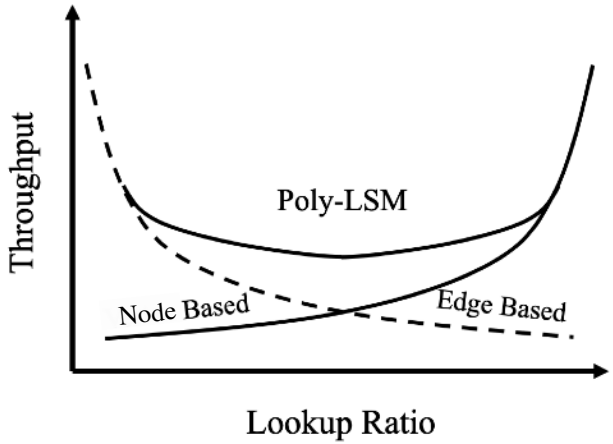
INDEX

- Introduction
- Background: LSM-Tree
- **Method: Poly-LSM**
- **Experiments & Summary**
- **Appendix**

Method: Poly-LSM

❑ Poly-LSM — Adapt Between the Two LSM Layouts

- Choose the update method adaptively, edge-based or node-based
- Higher throughput than node-only or edge-only across workload
- Built on RocksDB — Facebook's LSM-based key-value store



Method: Poly-LSM

❑ Two Entry Types — Pivot & Delta

- Pivot: holds most of the node's neighbors in one entry (≈ node-based)
- Delta: holds just one neighbor in one entry (≈ edge-based)
- In Poly-LSM, one node has a single pivot entry + several delta entries
  - A page (**pivot**) is neatly organized — easy to read
  - A sticky note (**delta**) just records one change — easy to write
  - Once in a while, notes get merged (**compaction**)

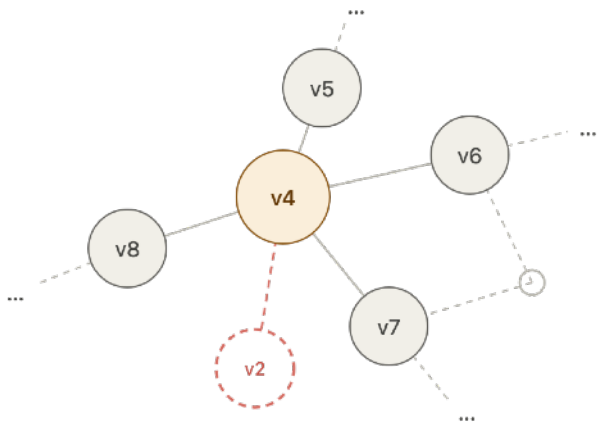
Pivot (node-based)

Delta (edge-based): paste onto pivot

Method: Poly-LSM

❑ How It Works — Update

- Every update decomposes into edge-level entries — edge update is the default
- Modification is also decomposed into edge-level operations



Adding v4 and connecting to neighbors

Add node (empty pivot)

v4-{}

Add edge (pivot or delta)

v4-v5 v4-v6 v4-v7 v4-v8

(undirected → also v5-v4, v6-v4, v7-v4, v8-v4)

Delete node (delta tombstone)

v4 ×

v5-v4 × v6-v4 × v7-v4 × v8-v4 ×

one node tombstone covers all of these

Delete edge (delta tombstone)

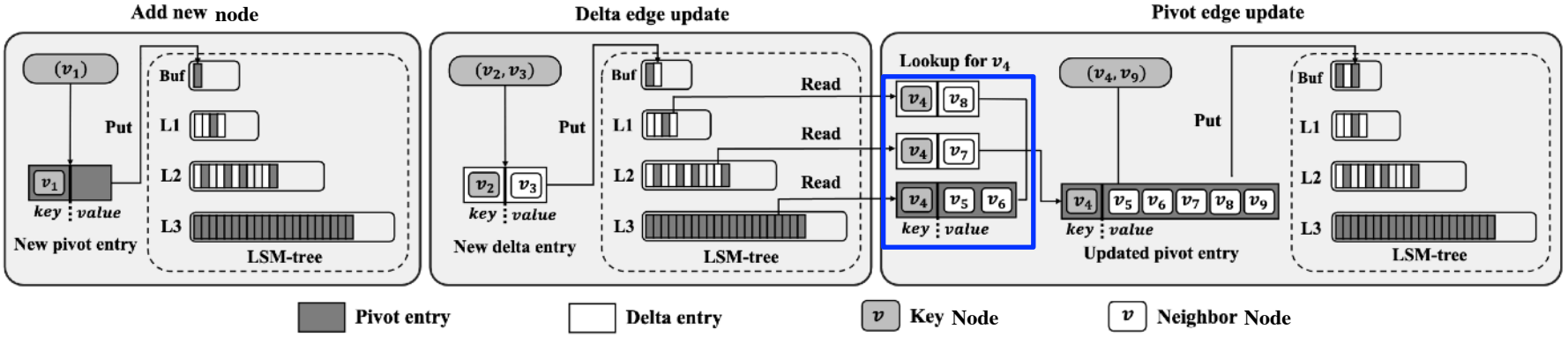
v4-v2 ×

(undirected → also v2-v4 ×)

Method: Poly-LSM

❑ How It Works — Update

- Add a node → **first** insert an empty pivot entry (every node starts with one pivot)
- Add an edge → select to appends as a **delta** (one entry) or as a **pivot** (read the disk, rewrite it whole)
- Delete → insert an entry marked "removed" (delta-like)
- As in any LSM-Tree (stack writes, don't edit) — every change is one new entry

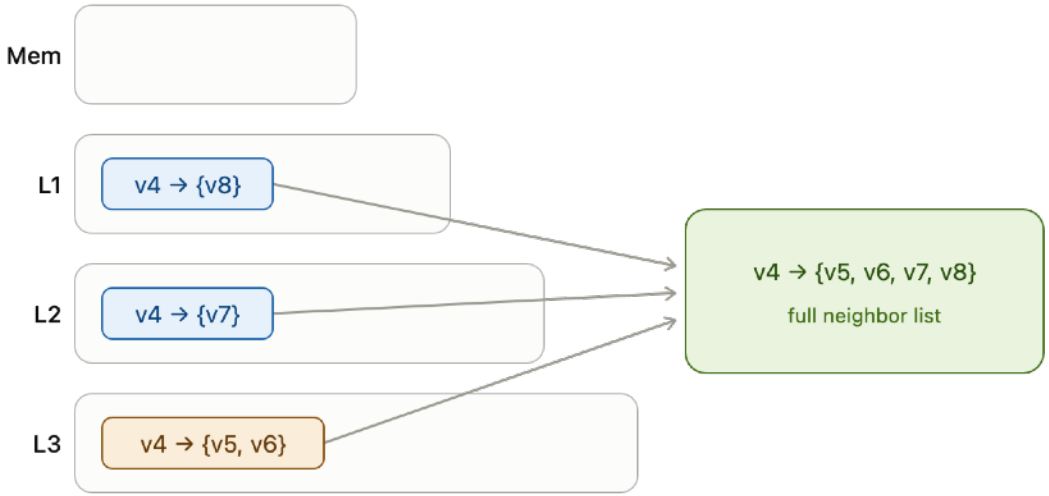


Scan all the levels and get deltas and pivot

Method: Poly-LSM

❑ How It Works — Lookup

- A node's neighbors are scattered: a few deltas on top, one pivot at the bottom
- Go down the levels, collect the deltas, stop the moment you hit the pivot

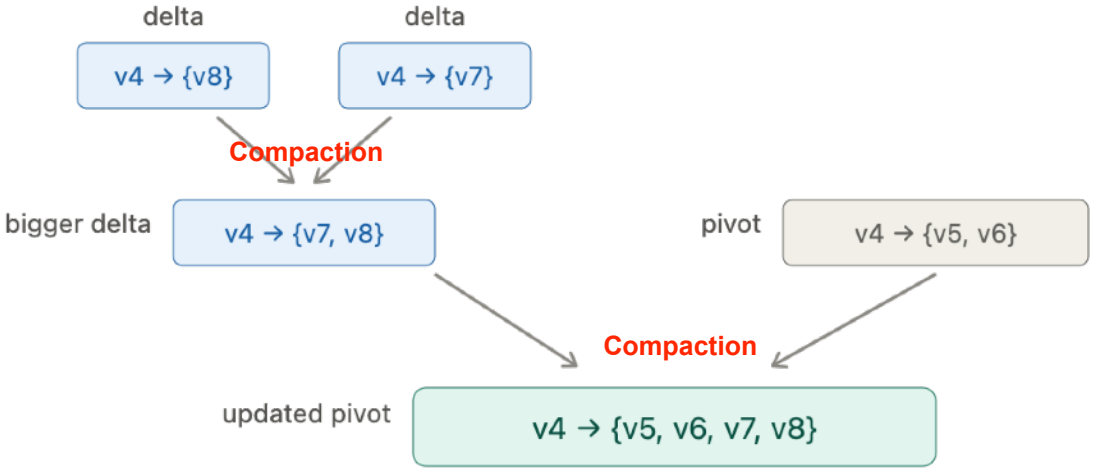


Method: Poly-LSM

❑ Poly-LSM's Core Idea — Hybrid Layout

- **Two entry types:** delta (≈ edge-based) and pivot (≈ node-based)
  - Stored deltas keep merging through compaction — eventually into the pivot → easy reads
- **Adaptive update:** Poly-LSM **adaptively** picks the update method — delta or pivot

➡ **Then, how do know we which update method to choose?**

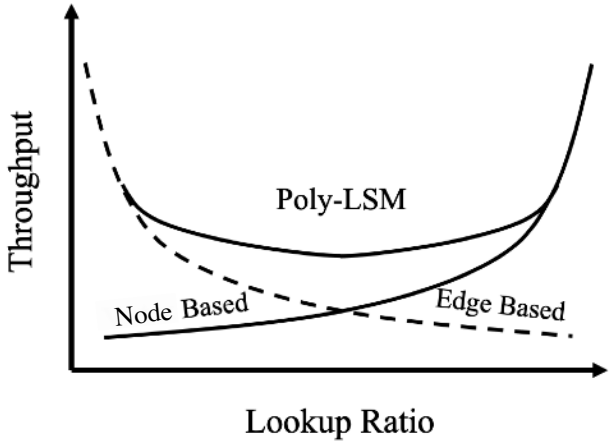


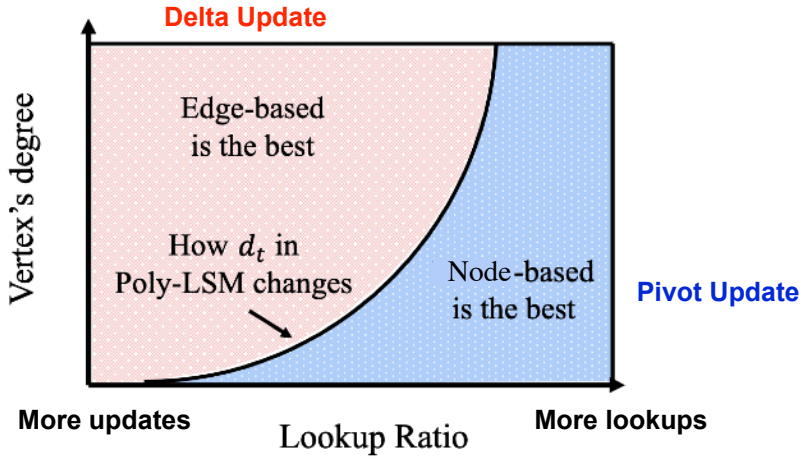
Method: Poly-LSM

❑ What Decides the Update Method — Degree & Workload

- **High degree** → delta (its huge list is costly to rewrite), **Low degree** → pivot (small list, fast reads)
  - Pivot is costly to write, delta to read — so if updates are rare, even high degree prefers pivot
- **More lookups** → boundary up (more pivot), **More updates** → boundary down (more delta)

➡ **Then, how is this boundary decided? — By the cost of delta update vs pivot update**





Method: Poly-LSM

❑ Cost of Each Update (Lifetime Block I/Os per Edge)

- **Poly-LSM** compares their **lifetime update cost** and picks the cheaper update method
- Inside LSM, both cut random access — so they differ in how much they read/write (block I/Os)
- Count the **total block I/O over its lifetime**, then compare and pick the cheaper update method
  - Counts **write** and **future reads** of delta update and pivot update

$$C_D = \frac{2I \cdot T \cdot L}{B} + \frac{\theta_L \cdot \bar{d}}{\theta_U \cdot (T-1)} \quad \begin{matrix} > \\ < \end{matrix} \quad C_P = 2 + \frac{(d(u)+1) \cdot I}{B} + \frac{(d(u)+2) \cdot I \cdot T \cdot L}{B}$$

Pivot update

Delta update

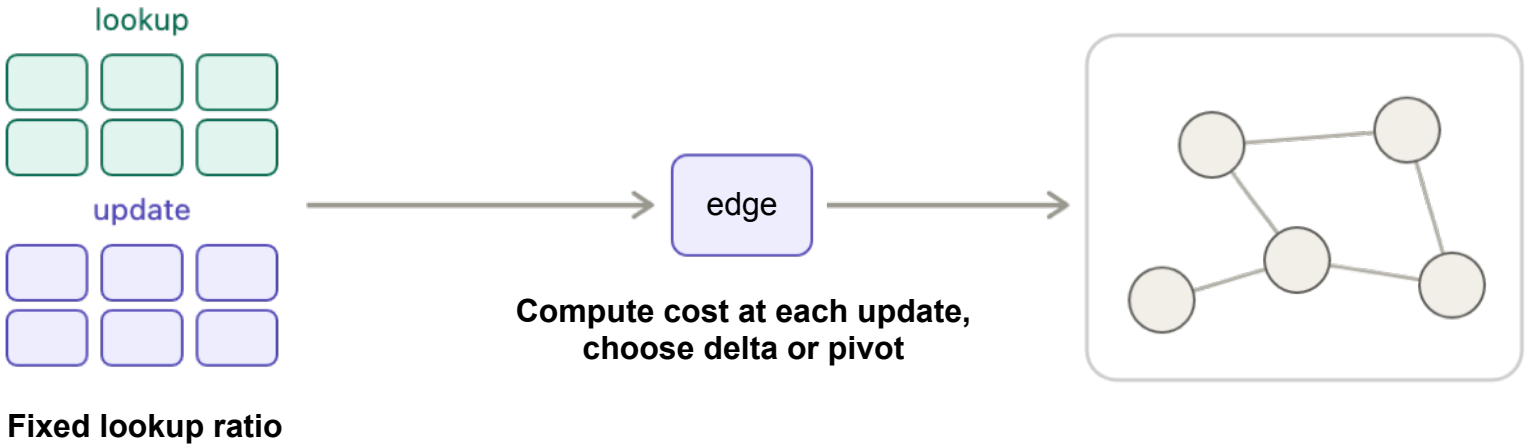
Lifetime cost of a delta

Lifetime cost of a pivot

Method: Poly-LSM

❑ Cost of Each Update (Lifetime Block I/Os per Edge)

- Assume a fixed lookup ratio across the workload (θ_L : θ_U)
- At each edge update, compute its lifetime cost and choose delta or pivot



Method: Poly-LSM

❑ Lifetime Cost of One Delta Edge Update

- Delta has two lifetime update cost of **one edge** — Update Cost & Lookup penalty
- Cost of expected writes of the delta until it merges into the pivot
- Lookup penalty: the extra lookup cost this delta imposes on other nodes

$$m \approx m + \frac{m}{T} + \frac{m}{T^2} + \frac{m}{T^3} + \cdots$$

Approximate number of entries at each level (log scale)

Write I/O Cost

Prospective Read I/O Cost

$$C_D = \frac{2I \cdot T \cdot L}{B} + \frac{\theta_L \cdot \bar{d}}{\theta_U \cdot (T-1)}$$

$$\text{lifetime} \approx \frac{m}{T} + \frac{m}{T^2} + \frac{m}{T^3} + \cdots = \frac{\frac{m}{T}}{1 - \frac{1}{T}} = \frac{m}{T-1}$$

To reach the pivot, only the levels above the last level need to be filled.

I: size of one node ID
T: compactions to reach the next level
L: number of levels to pass through
T·L: total rewrites
B: block size

$$\frac{m}{T-1} \times \frac{\theta_L}{\theta_U} \times \frac{1}{n}$$

m: total number of edges
m/(T−1) : updates needed to reach the pivot
θ_L: lookup ratio
θ_U: update ratio
θ_L/θ_U: lookups per update
1/n: chance a lookup targets this delta's node
$\bar{d}$: average degree

Method: Poly-LSM

❑ Lifetime Cost of One Delta Edge Update

- Delta has two lifetime update cost of **one edge** — Update Cost & Lookup penalty
- Cost of expected writes of the delta until it merges into the pivot
- Lookup penalty: the extra lookup cost this delta imposes on other nodes

Write Block I/Os until it merge with pivot

$$\frac{2I \cdot T \cdot L}{B}$$

I: size of one node ID
T: the level size ratio, compactions needed to reach the next level
L: number of levels to pass through, pivot is usually in the located at the largest level
T·L: total rewrites
B: block size

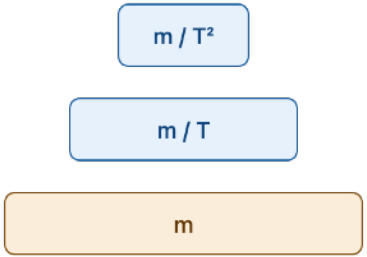
Method: Poly-LSM

❑ Lifetime Cost of One Delta Edge Update

- Delta has two lifetime update cost of **one edge** — Update Cost & Lookup Cost
- Cost of expected writes of the delta until it merges into the pivot
- Lookup penalty: the extra lookup cost this delta imposes on other nodes

$$m \approx m + \frac{m}{T} + \frac{m}{T^2} + \frac{m}{T^3} + \cdots$$

$$\text{lifetime} \approx \frac{m}{T} + \frac{m}{T^2} + \frac{m}{T^3} + \cdots = \frac{\frac{m}{T}}{1 - \frac{1}{T}} = \boxed{\frac{m}{T-1}}$$



$$\boxed{\frac{m}{T-1}} \times \frac{\theta_L}{\theta_U} \times \frac{1}{n}$$

m: total number of edges
m/(T−1) : updates needed to reach the pivot
θ_L/θ_U: lookups per update
1/n: chance a lookup targets this delta's node
$\bar{d}$: average degree (m/n)

Prospective Read I/O Cost

$$\frac{\theta_L \cdot \bar{d}}{\theta_U \cdot (T-1)}$$

Method: Poly-LSM

❑ Lifetime Cost of One Pivot Edge Update

- Pivot has one lifetime update cost of **one edge** — Update Cost
- Cost of fetching the pivot and rewriting the whole

[block1][block2][block3]
....[=entry=][=entry=]....

Misalign

Lookup Cost when update

$$C_P = 2 + \frac{(d(u)+1)\cdot I}{B}$$

2: fixed read cost
 * **1**: finding the delta, most at L-1, so counted as 1
 * **1**: pivot doesn't align to a block boundary
d(u): the degree (number of neighbors)
d(u)+1: neighbors + key
I: size of one node ID
B: block size

Rewrite I/O Cost (pivot update & compaction)

$$+ \frac{(d(u)+2)\cdot I \cdot T \cdot L}{B}$$

d(u)+2: neighbors + key + new ID
I: size of one node ID
T: compactions to reach the next level
L: number of levels to pass through
T·L: total rewrites
B: block size

Method: Poly-LSM

❑ Compare Lifetime Cost, Pick the Cheaper

- **Poly-LSM** compares their **lifetime update cost** and picks the cheaper update method
- Pivot lifetime update cost ∝ degree d(u)
- Delta lifetime update cost ∝ workload θ_L/θ_U

Pivot update

$$C_D = \frac{2I \cdot T \cdot L}{B} + \frac{\theta_L \cdot \bar{d}}{\theta_U \cdot (T-1)} \;\; > \;\; < \;\; C_P = 2 + \frac{(d(u)+1) \cdot I}{B} + \frac{(d(u)+2) \cdot I \cdot T \cdot L}{B}$$

Delta update

Lifetime cost of a delta

Lifetime cost of a pivot

Method: Poly-LSM

❑ A Graph DB Powered by Poly-LSM — Aster

- A complete graph database that uses Poly-LSM as its storage engine
- Supports the Gremlin query language for a wide range of graph tasks

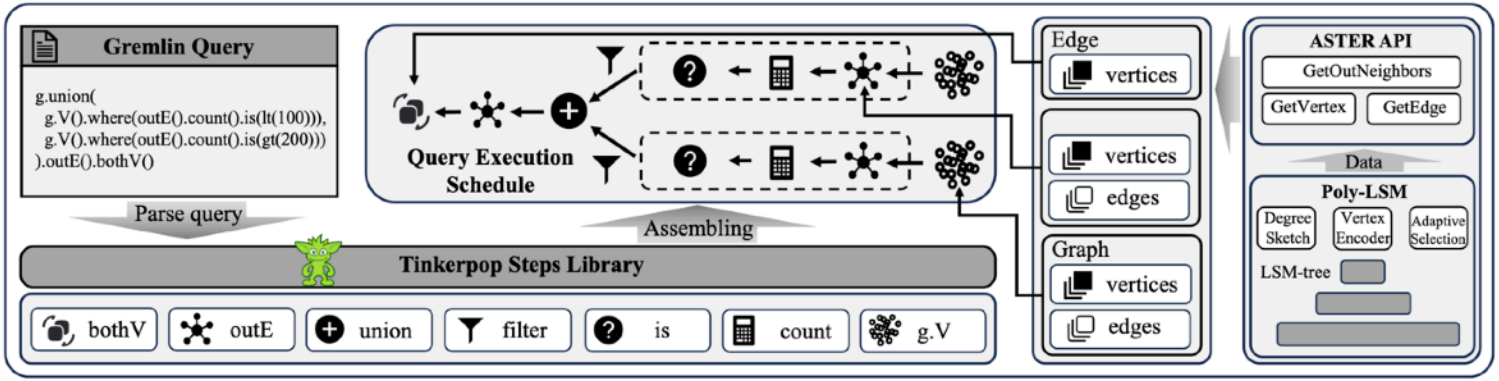


Fig. 5. The architecture of ASTER.

INDEX

- Introduction
- Background: LSM-Tree
- Method: Poly-LSM
- **Experiments & Summary**
- **Appendix**

Experiments

❑ Datasets & Experiment Method

- Focus on two representative operations: adding edges and retrieving neighbors
- From an empty DB, insert edges and lookup neighbors at the target ratio, measuring throughput

Scale	Graph	n	m	$\bar{d}$	Size
Moderate	DBLP [92]	317,080	1,049,866	3.31	131MB
	Twitch [79]	168,114	6,797,557	40.43	213MB
	LDBC Datagen* [57]	184,329	767,894	4.17	22MB
	Wiki-Talk	2,394,385	5,021,410	2.10	140MB
	Cit-Patents	3,774,768	16,518,947	4.38	351MB
Large	Wikipedia [57]	3,333,397	123,709,902	37.11	1.8GB
	Orkut [92]	3,072,441	234,370,166	76.28	1.3GB
	Freebase Large* [57]	28,408,172	31,475,362	1.11	1.4GB
Massive	Twitter [49]	41,652,230	1,202,513,046	57.74	24GB

Table 3. Datasets. n is the total number of vertices; m is the number of edges; $\bar{d}$ represents the average degree. The datasets with * are property graphs.

Experiments

- **Throughput vs. Baselines Across Datasets**

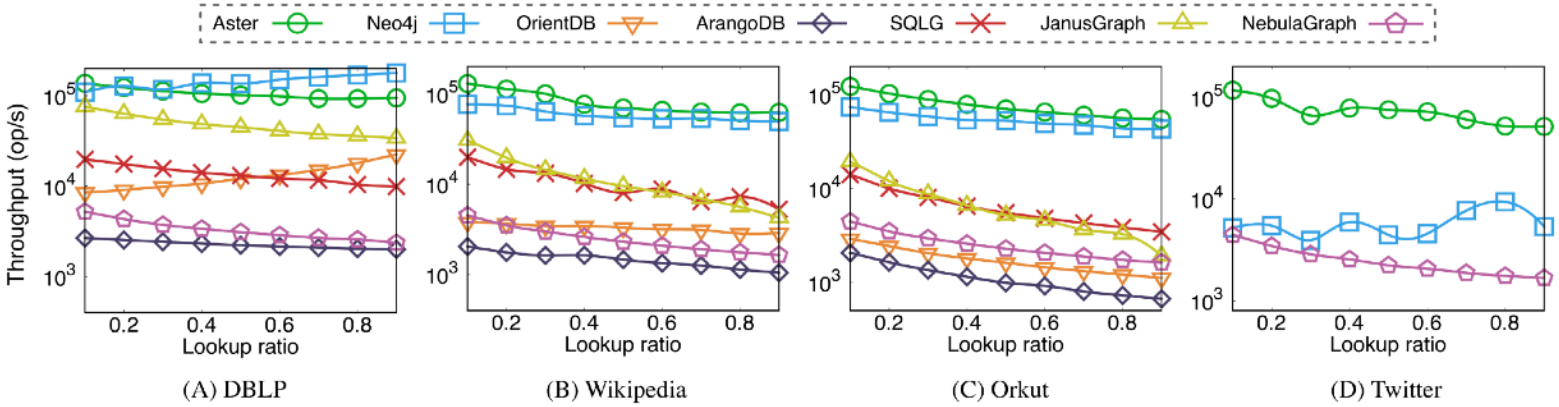


Fig. 6. ASTER demonstrates strong efficiency and scalability across various workloads and dominates all baselines on large-scale and massive-scale graph datasets.

Experiments

❑ Throughput Across Varying Workloads

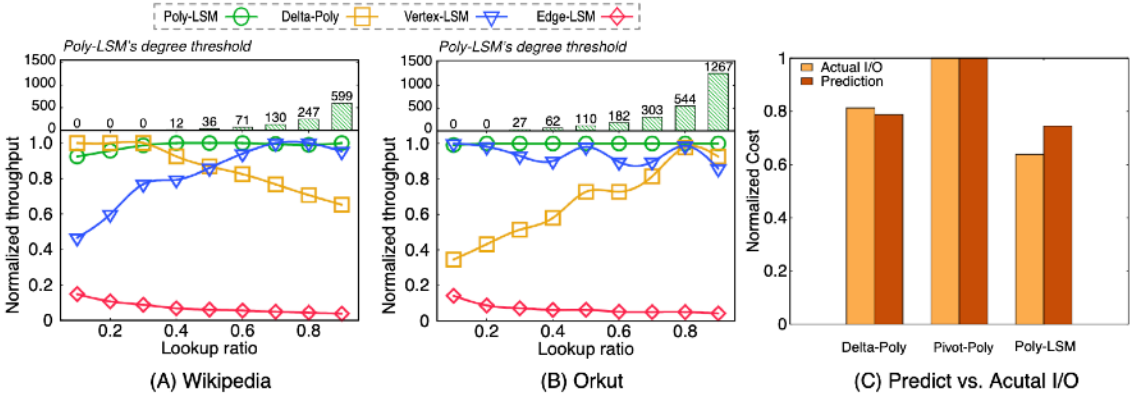


Table 5. The statistics of ASTER on Wikipedia under 50% lookup ratio workload.

Statistics of Poly-LSM	
CPU Time of Adaptive Mechanism	0.723 us / op
Morris Counter Read	0.107 us / op
Read Block Latency	10345.114 ms
Write Block Latency	572.054 ms
Number of Vertex-based Updates	879286
Number of Edge-based Updates	1120714
Avg. Degree of Vertex-based Update	3.58
Avg. Degree of Edge-based Update	118.36

Fig. 8. (A) and (B) shows POLY-LSM exhibits strong robustness as the workload changes and outperforms other LSM-tree-based baselines. (C) indicates the cost model of POLY-LSM accurately predicts the I/O overhead.

Summary

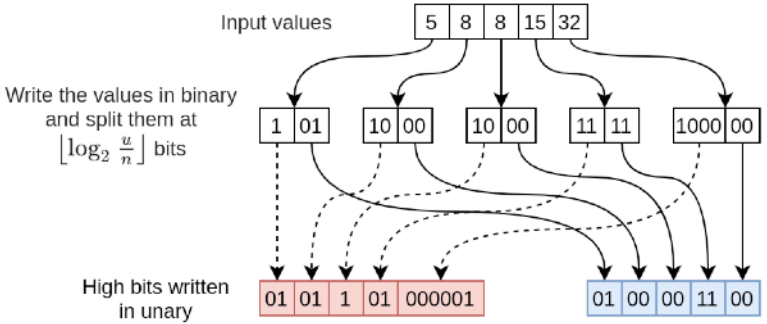
- Aster — graph database for modern evolving graphs
- Modern evolving graphs need fast update, fast lookup, and scalability
- Linked-list & Relational tables suffer random access, even LSM-tree has a node vs. edge trade-off
- Poly-LSM: adaptively picks the update method, achieving all three at once
- Results: best throughput across all workloads, and up to 17× on the billion-edge Twitter graph

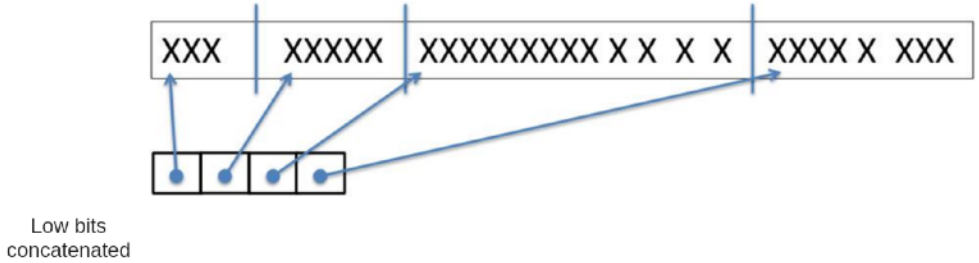
Thank you!

Appendix

❑ Partitioned Elias-Fano Encoding

- Storing all neighbor IDs explicitly can be space-expensive
- **Elias–Fano Encoding**: a compact encoding for a sorted integer list
- A node's neighbor list is naturally a sorted list of neighbor IDs, so EF fits well
- Real graphs are skewed and exhibit locality, so a single global encoding can waste space
- **Partition the neighbor list and apply EF per partition for better compression**





Appendix

❑ Degree Sketch

- Adaptive updates need node degree, but per-update degree lookups incur I/O overhead
- In-memory approximate sketch based on Morris Counter
  - 8-bit sketch per node: 4-bit exponent (high bits) + 4-bit mantissa (low bits)
  - On each degree increment, mantissa increases probabilistically with probability $1 / 2^{(\text{exponent})}$
  - If mantissa increments at 15, it resets to 0 and exponent increases by 1
- **An in-memory 8-bit sketch avoids extra I/O and approximates node degrees accurately**